MAC5921 - Deep Learning — EP1

André Nogueira Ribeiro

October 25, 2026

1 Declaração sobre uso de IA

O código implementado para este trabalho foi gerado com ajuda da IA, utilizando uma IDE com modelos integrados. Abaixo se encontra com mais detalhes como foi utilizada a ajuda desta ferramenta:

- Códigos associados às implementações de funções e orquestrações para rodar os experimentos foram feitos com muito cuidado, o que inclui revisões detalhadas das sugestões geradas por IA, discussões com os modelos sobre diferentes possibilidades e razões para certas sugestões.
- Códigos de teste foram revisados de maneira um pouco mais superficial, conferindo se o que estava sendo testado fazia sentido e em alguns casos indo mais a fundo no como isso era feito.
- Códigos para geração de figuras e tabelas com os resultados foram feitos com pouca intervenção da minha parte.
- Este relatório foi feito sem nenhuma ajuda de modelos, exceto para pequenas formatações e sintaxe de LaTeX.

2 Código

O código, incluindo este relatório feito em LaTeX, pode ser encontrado em:

- https://github.com/deconr/MAC5921-DeepLearning.

3 Experimentos

Essa seção descreve os experimentos realizados (incluindo o planejamento) e apresenta os resultados obtidos. Analisarei o valor da função de perda e da acurácia para os conjuntos de treinamento e validação ao longo do treinamento, além do resultado no conjunto de teste. O dataset utilizado nos experimentos foi o MNIST.

Nesse contexto, é importante mencionar que, para o conjunto de treinamento, o valor final é uma média dos valores que são calculados para cada batch com os pesos atuais da rede (atualizados após cada batch), ou seja, cada valor de loss/acc é calculado para rede com pesos diferentes dentro de uma mesma época. Já para o conjunto de validação, o valor é calculado com os pesos congelados ao final da época. Isso explica o porquê os gráficos referentes ao treinamento iniciam com valores piores em relação a métrica avaliada. Por exemplo, no gráfico Loss / Epoch, o valor inicial das curvas de treinamento são bem maiores que o valor inicial das curvas de validação.

3.1 Comparação base entre NN e CNN

A ideia desse experimento é comparar NNs com CNNs, considerando aspectos gerais, sem modificar o dataset utilizado. Para isso, construí modelos com números similares de parâmetros e analisei o comportamento durante o treinamento e validação (results/curves.png). O resultado nos dados de teste também será mencionado.

Análise	**Treinamento**	**Validação**	**Considerações**
Variação da loss ao longo das épocas (curves.png). CNN com 52,138 parâmetros. NN com 52,480 parâmetros Mais detalhes da arquitetura no arquivo models.py.	Cai muito nas duas primeiras épocas, o que é esperado dado que os pesos da rede começam com valores aleatórios. Percebe-se que já na primeira época a CNN associa-se a uma loss menor em relação a NN, o que continua sendo verdade por todo o treinamento. Também é possível perceber que a CNN converge mais rápido, o que reflete uma inclinação menor mais rapidamente quando comparada a curva da NN.	Cai menos do que as curvas de treinamento nas primeiras épocas (devido ao que foi explicado acima), porém a comparação também mostra que a CNN se associa a uma loss menor em relação a NN por todo o treinamento. A diferença é que na validação as curvas atingem um valor estável (considerando as épocas treinadas) mais rapidamente porém para um valor mais alto. Um detalhe importante é que a curva para validação pode piorar mesmo quando a curva de treinamento segue melhorando, o que acontece mais evidentemente na curva referente à NN. Isso se da provavelmente pois o treinamento está overfitando.	As análises do resultado confirmam o que já era esperado: a CNN teve um desempenho melhor, o que normalmente é atribuído ao fato de que a rede leva em consideração a localidade (convoluções que analisam pixels vizinhos), o que é uma característica dos dados analisados (imagens). Concordando com o que foi mencionado, a acurácia final no conjunto de teste também foi melhor para CNN, 0.9879 contra 0.9728 (summary.json)

Vale ressaltar que uma análise análoga (porém espelhada em relação a horizontal) pode ser feita para a acurácia. Um aspecto que as difere é o fato da loss levar em consideração a confiança enquanto a acurácia analisa apenas se o modelo acertou ou errou (isso faz com que os gráficos não sejam exatamente espelhados).

3.2 Ruído gaussiano no conjunto de teste

O objetivo desse experimento é analisar a robustez dos modelos em relação a ruídos. A ideia é medir o quanto a acurácia cai quando adicionamos ruído nas imagens do conjunto de teste.

O ruído é adicionado a cada pixel de forma independte seguindo uma distribuição normal com média zero e desvio padrão variável (utilizei $\sigma = 0,2$ e $\sigma = 0,3$). O ruído é adicionado nos dados depois de transformá-los para float e antes da normalização, quando estão no intervalo [0,1]. Vale ressaltar que os dados são mantidos no intervalo [0,1] mesmo que o ruído faça os dados extrapolarem esse limite. Adicionei uma imagem com um sample de imagens após o ruído para possibilitar a visualização em cada caso (e.g samples_test.png).

É importante ressaltar que os gráficos de treinamento e validação continuam iguais ao do experimento "Comparação base entre NN e CNN" dado que o ruído é acrescentado apenas no conjunto de teste.

Abaixo estão as conclusões do experimentos baseado nos resultados contidos nos diretórios test_noise_gaussian_std0.2 e test_noise_gaussian_std0.3. A acurácia no conjunto de testes está no campo `test_acc` dos arquivos `summary.json`.

Comportamento da CNN em relação a própria CNN no experimento "Comparação base entre NN e CNN": Utilizando um ruído com parâmetro 0.2, a acurácia cai de 0.9879 para 0.9832 demonstrando uma resiliência da rede a pequenos rúidos. Com o parâmetro do ruído igual a 0.3 a acurácia cai para 0.9548, demonstrando um gap maior porém ainda com um valor de acurácia alto.

Comportamento da NN em relação a própria NN no experimento "Comparação base entre NN e CNN": No caso da NN percebe-se uma queda na acurácia de 0.9728 para 0.9481 no caso de menos rúido. No caso de maior ruído a acurácia vai para 0.8794.

Aspectos gerais comparando CNN e NN: Concordando com os resultados apresentados, percebe-se que a CNN possui uma robustez maior em relação a adição de ruído. Com ruído "0.2", a acurácia cai 0.48% enquanto para NN cai 2.54%. Já com o ruído "0.3", a acurácia cai 3.35% para CNN e 9.6% para a NN.

3.3 Translação das imagens no conjunto de teste

O objetivo desse experimento é analisar a robustez dos modelos em relação a translação das imagens. A ideia é medir o quanto a acurácia cai quando transladamos as imagens do conjunto de teste.

3.4 Observações adicionais

Em geral, olhando para os arquivos "summary.json" dos experimentos, a CNN demorou mais para treinar. Isso faz sentido dado que na CNN, um mesmo peso é utilizado por entradas diferentes do input (no caso de imagens, por pixels diferentes da imagem), enquanto na NN um peso multiplica apenas um pixel da imagem (considerando a primeira camada).